

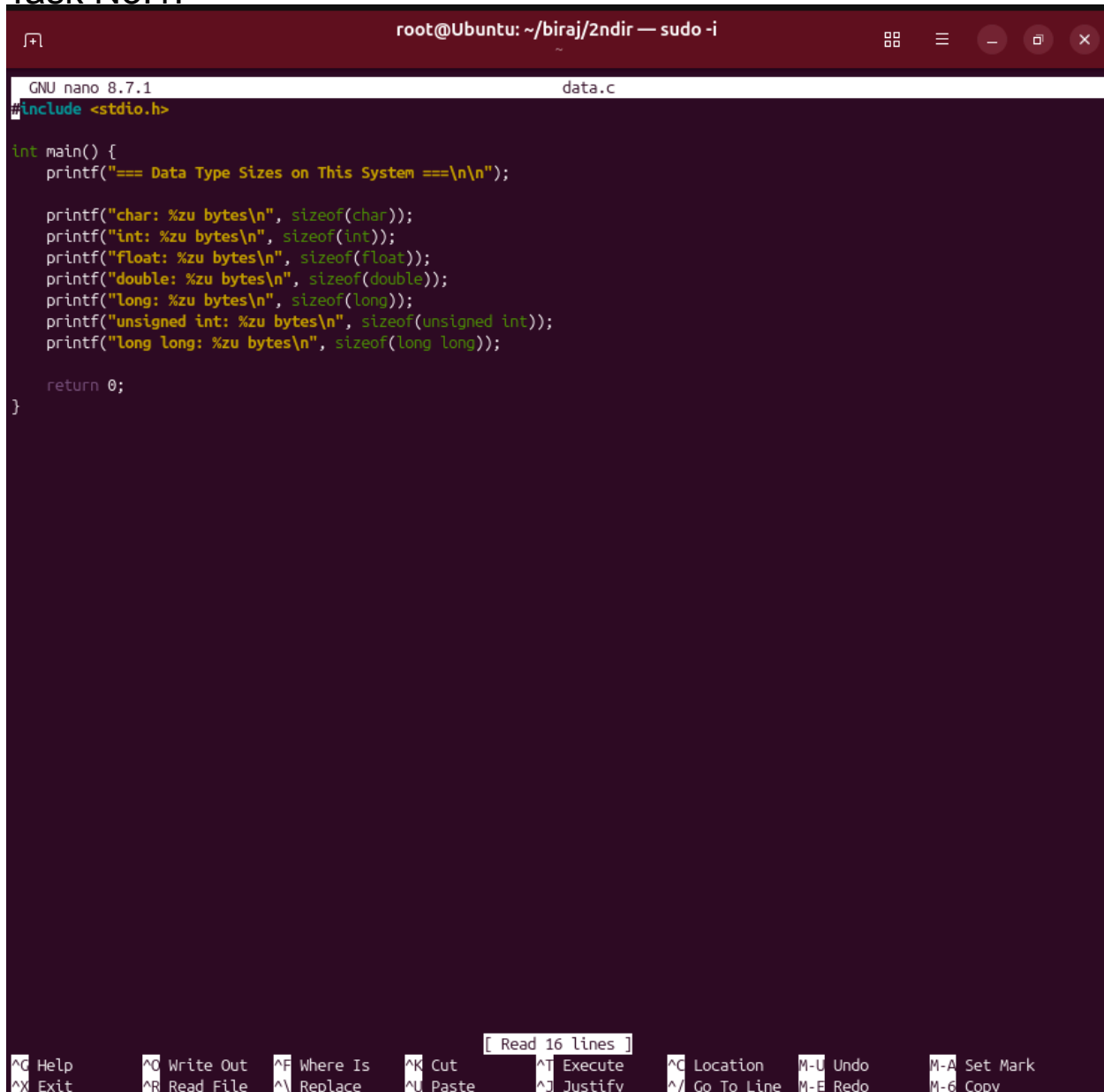
Table of Content:

Task No.1:	2
Task No.2:	3
Knowledge Questions.	6

Table of figures:

Figure: Program to observe data type sizes.	3
Figure: Correct pointer and heap usage.	4
Figure: Correct pointer and heap usage continue.	5
Figure: Compilation and execution of the data2.c	5

Task No.1:



The screenshot shows a terminal window with the title bar "root@Ubuntu: ~/biraj/2ndir — sudo -i". The terminal is running the GNU nano 8.7.1 editor, editing a file named "data.c". The code in the file is as follows:

```
GNU nano 8.7.1 data.c
#include <stdio.h>

int main() {
    printf("=== Data Type Sizes on This System ===\n\n");

    printf("char: %zu bytes\n", sizeof(char));
    printf("int: %zu bytes\n", sizeof(int));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));
    printf("long: %zu bytes\n", sizeof(long));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("Long long: %zu bytes\n", sizeof(long long));

    return 0;
}
```

The bottom of the terminal shows the nano editor's command palette with the following options:

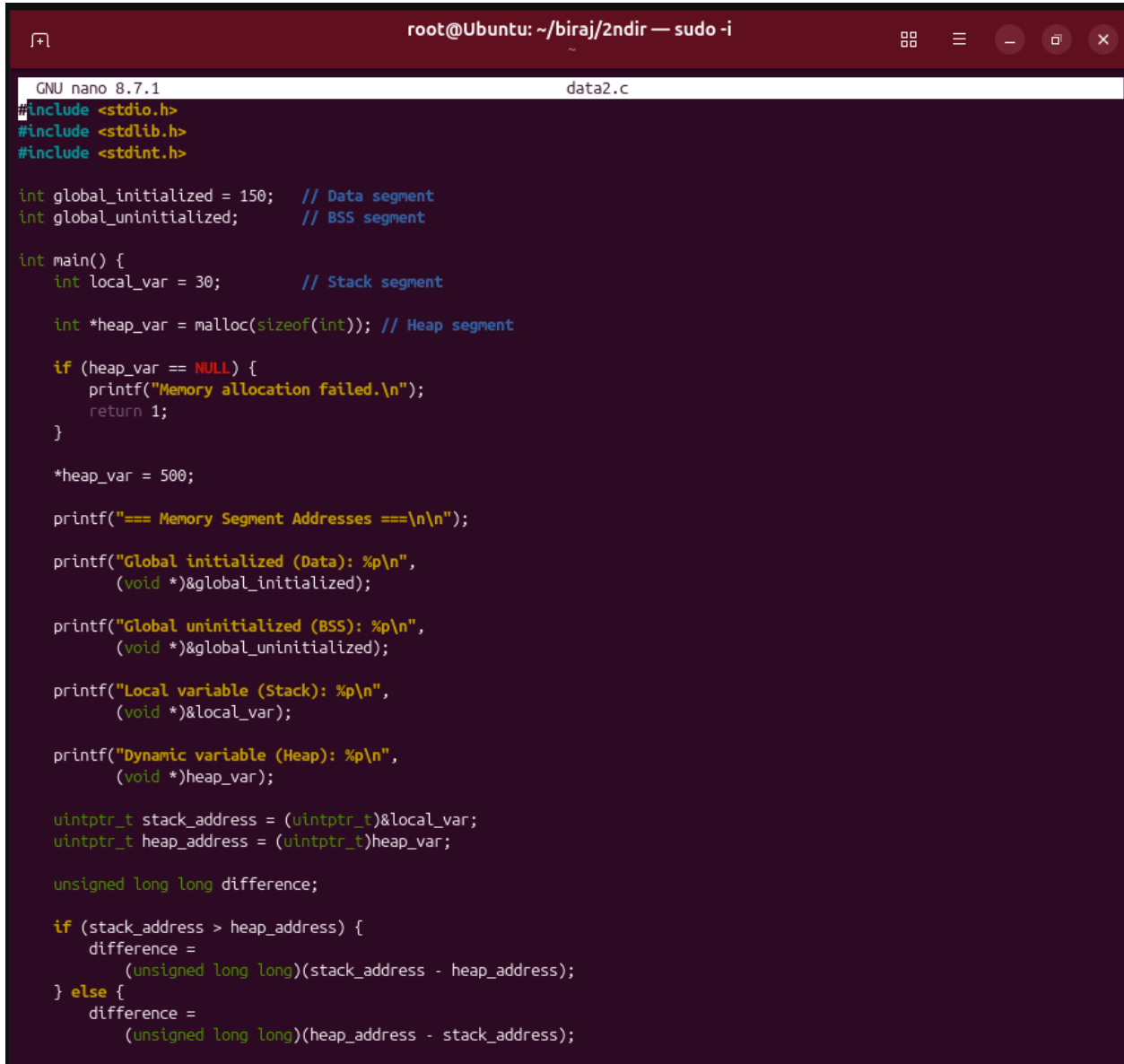
^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo	M-A Set Mark
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo	M-C Copy

A status bar at the bottom indicates "[Read 16 lines]".

Figure: Program to observe data type sizes.

Summary: The program uses `sizeof()` to determine the memory size of each data type and compares the results with the lab's architecture table. On a typical 64-bit Linux system, sizes range from 1 byte for `char` to 8 bytes for `double`, `long`, and `long long`.

Task No.2:



```
GNU nano 8.7.1 data2.c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

int global_initialized = 150; // Data segment
int global_uninitialized; // BSS segment

int main() {
    int local_var = 30; // Stack segment

    int *heap_var = malloc(sizeof(int)); // Heap segment

    if (heap_var == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    *heap_var = 500;

    printf("=== Memory Segment Addresses ===\n\n");

    printf("Global initialized (Data): %p\n",
        (void *)&global_initialized);

    printf("Global uninitialized (BSS): %p\n",
        (void *)&global_uninitialized);

    printf("Local variable (Stack): %p\n",
        (void *)&local_var);

    printf("Dynamic variable (Heap): %p\n",
        (void *)heap_var);

    uintptr_t stack_address = (uintptr_t)&local_var;
    uintptr_t heap_address = (uintptr_t)heap_var;

    unsigned long long difference;

    if (stack_address > heap_address) {
        difference =
            (unsigned long long)(stack_address - heap_address);
    } else {
        difference =
            (unsigned long long)(heap_address - stack_address);
    }
}
```

Figure: Correct pointer and heap usage.

```
        (unsigned long long)(heap_address - stack_address);
    }

    printf("\nAddress difference between Stack and Heap: %llu bytes\n",
        difference);

    free(heap_var);

    return 0;
}
```

Help Write Out Where Is Cut Execute Location Undo Set Mark
Exit Read File Replace Paste Justify Go To Line Redo Copy

Figure: Correct pointer and heap usage continue.

```
root@Ubuntu: ~/biraj/2ndir — sudo -i

root@Ubuntu:~/biraj/2ndir# nano data2.c
root@Ubuntu:~/biraj/2ndir# nano data2.c
root@Ubuntu:~/biraj/2ndir# gcc data2.c -o data2
root@Ubuntu:~/biraj/2ndir# ./data2
=== Memory Segment Addresses ===

Global initialized (Data): 0x636da0eb2010
Global uninitialized (BSS): 0x636da0eb2018
Local variable (Stack): 0x7ffc00287be4
Dynamic variable (Heap): 0x636dc8bbe010

Address difference between Stack and Heap: 31397140798420 bytes
root@Ubuntu:~/biraj/2ndir#
```

Figure: Compilation and execution of the data2.c

Summary: The addresses show that local variables are stored on the stack, dynamic data on the heap, and global variables in Data/BSS. Addresses may vary due to virtual memory and address randomization. Typically, the stack is at higher addresses, while Data/BSS and heap occupy lower and middle regions.

Knowledge Questions.

1) What is a data type in programming and operating systems?

A data type defines what kind of data a variable can store, such as int, float, char, or double, and determines its size and how it is interpreted in memory.

2) Name the five main memory segments in a process memory layout.

Text (Code), Data, BSS, Heap, and Stack.

3) What is the difference between the Data segment and BSS segment?

The Data segment stores initialized global/static variables, while the BSS segment stores uninitialized or zero-initialized global/static variables.

4) Why does the OS place the stack at high virtual addresses and the text segment at low addresses?

It allows the stack to grow downward and the heap to grow upward, providing efficient use of the available virtual address space.

5) What happens to stack memory when a function returns? Who reclaims it?

The function's stack frame is removed when it returns, and the stack pointer moves back, effectively reclaiming the memory automatically.

6) How does the OS know which memory belongs to which process?

The OS uses page tables and virtual memory mappings to keep each process's memory separate and identify which physical memory belongs to it.